Proposal

Introduction/Problem Statement

Context/Background

- What is electrochemical reactor chemical modeling
  - Why is it important
  - Show how it is very complex
  - There are libraries like eCherry
    - Electrochemical reactor simulation frameworks such as eCherry enable modular modeling of electrochemical systems in Modelica
    - However Construction requires manual assembly

Problem

- Constructing reactor simulations requires manually assembling multiple low-level components and configuring simulation parameters
- Domain experts may lack Modelica expertise
  - Primarily trained in electrochemistry rather than simulation modeling
  - Making this process complex and time consuming
  - Prominent for researchers and engineers without specifically modeling expertise

Proposed Solution

- This project explores a model-driven approach for electrochemical reactor design by introducing a domain-specific modeling language (DSL) that allows users to describe reactor configurations at a higher level of abstraction
  - Models specified in the DSL are automatically transformed into eCherry Modelica simulation models

- The goal is to simplify reactor modeling workflows while demonstrating key concepts of model-driven software development, including metamodeling, model transformation, and automated artifact generation

Example Text Excerpt (AI):

Electrochemical reactor simulation frameworks such as **eCherry** enable modular modeling of electrochemical systems using the Modelica language. These frameworks allow engineers and researchers to construct simulation models by assembling reusable components representing electrodes, electrolytes, flow channels, and other reactor elements. However, constructing such models requires manually assembling multiple low-level Modelica components and configuring simulation parameters. This process can be complex and time-consuming, particularly for domain experts who are primarily trained in electrochemistry rather than simulation modeling.

Model-driven engineering (MDE) addresses similar challenges in software development by raising the level of abstraction used to describe systems and automatically generating executable artifacts from high-level models. In this project, we explore a model-driven approach for electrochemical reactor design by introducing a **domain-specific modeling language (DSL)** that allows users to describe reactor configurations at a higher level of abstraction. Models specified in the DSL are automatically transformed into **eCherry-compatible Modelica simulation models**.

The goal of this project is to simplify reactor modeling workflows while demonstrating key concepts of model-driven software development, including **metamodeling, model transformation, and automated artifact generation**.

Proposed Approach

- The proposed system follows a **model-driven engineering workflow** in which high-level reactor descriptions are automatically transformed into executable eCherry simulation models. The approach introduces a domain-specific modeling layer that abstracts away low-level Modelica component configuration while preserving compatibility with the eCherry simulation framework.

- The modeling pipeline consists of 4 main stages:

```
DSL Reactor Model
        ↓
Metamodel-based Representation
        ↓
Model Transformation
        ↓
Generated eCherry Modelica Simulation Model
```

Metamodel

The system will define a **metamodel** describing the structure of electrochemical reactor systems. The metamodel captures the key components and operating parameters commonly used in electrochemical reactor simulations.

Example entities in the metamodel include:

- Reactor
- Electrode
- Electrolyte
- Separator
- FlowChannel
- OperatingConditions

Operating conditions may include parameters such as temperature, pressure, and flow rate. The metamodel defines the valid relationships between these entities and serves as the structural foundation for the modeling language

Domain-Specific Language

- A **domain-specific language (DSL)** will be introduced to allow users to describe reactor configurations at a higher level of abstraction than the underlying Modelica implementation. The DSL will provide a concise textual syntax for defining reactor components and simulation parameters
- For example, a simplified reactor specification might appear as:

```
reactor Electrolyzer {
    electrode_area = 10 cm^2
    electrolyte = KOH
    temperature = 298K
}
```

- Each DSL specification represents a **model instance** conforming to the defined metamodel.

Model Transformation

- A transformation engine will translate DSL models into **eCherry-compatible Modelica code**
    - This process involves parsing the DSL description, constructing an internal model representation based on the metamodel, and applying transformation rules that map reactor components to the corresponding eCherry Modelica components
    - The generated Modelica models will assemble appropriate eCherry components and configure simulation parameters required for execution within a Modelica simulation environment

Generated Artifacts

- The final output of the transformation process is an **executable eCherry Modelica simulation model**
- These generated models can be simulated using standard Modelica tools supporting the eCherry library
- By automating this generation process, the system enables users to rapidly explore reactor configurations without manually constructing Modelica models

Example Text Excerpt (AI):

The proposed system follows a model-driven engineering workflow in which electrochemical reactor designs specified at a high level of abstraction are automatically translated into executable simulation models compatible with the eCherry framework. The goal is to reduce the complexity of manually constructing

Modelica-based simulation models while preserving the flexibility of the underlying modeling library.

The first component of the approach is the definition of a **metamodel** that captures the key structural elements of electrochemical reactor systems. The metamodel represents common reactor components and operating parameters typically used in electrochemical simulations. Core entities in the metamodel include reactors, electrodes, electrolytes, separators, and flow channels, along with operating conditions such as temperature, pressure, and flow rate. The metamodel defines the relationships between these entities and establishes the structure of valid reactor models within the system.

Based on this metamodel, a **domain-specific language (DSL)** will be designed to allow users to describe reactor configurations at a higher level of abstraction than the underlying Modelica implementation. Instead of manually assembling individual simulation components, users will specify reactor structures and parameters using concise DSL statements. Each DSL description represents a model instance conforming to the defined metamodel.

The system will then perform a **model transformation** that converts DSL models into eCherry-compatible Modelica simulation models. This process involves parsing the DSL specification, constructing an internal representation of the reactor model, and applying transformation rules that map DSL elements to the corresponding eCherry Modelica components. The transformation engine will automatically generate Modelica code that assembles the required simulation components and configures their parameters.

The output of this process is a fully defined **eCherry Modelica simulation model** that can be executed using standard Modelica simulation tools. By automating the generation of simulation models from higher-level descriptions, the system enables users to rapidly define and modify reactor configurations without manually constructing low-level Modelica models.

Implementation Plan

System Architecture

```
DSL parser
    ↓
internal model representation
    ↓
transformation engine
    ↓
Modelica code generator
```

- **DSL Parser**
  - Reads reactor descriptions written in the DSL
  - Extracts components, parameters, and operating conditions
  - Converts DSL input into structured model data
- **Internal Model Representation**
  - Stores parsed reactor models
  - Implements the defined **metamodel structure**
- **Model Transformation Engine**
  - Applies transformation rules to convert DSL models into simulation models
  - Maps DSL constructs to corresponding **eCherry Modelica components**
  - Ensures generated models follow valid Modelica structure
- **Modelica Code Generator**
  - Produces executable **eCherry-compatible Modelica code**
  - Assembles required simulation components
  - Configures parameters specified in the DSL

Tools and Technologies

The prototype system will likely use the following technologies:

- **Python**
  - DSL parsing

    - transformation logic
    - code generation
- **Text-based DSL**
    - simple syntax for defining reactor configurations
- **Modelica**
    - target simulation language
- **eCherry Modelica library**
    - simulation framework for electrochemical reactors
- **Modelica simulation environment**
    - used to run generated models

Example Text Excerpt (AI):

To evaluate the proposed model-driven approach, we will implement a prototype tool that automatically generates eCherry-compatible Modelica simulation models from high-level reactor descriptions. The prototype will demonstrate the full modeling pipeline, including DSL parsing, model transformation, and automatic code generation.

The system will include a DSL parser that interprets textual reactor descriptions written using the proposed modeling language. Parsed reactor models will be converted into an internal representation based on the defined metamodel, capturing the structure of reactor components and operating conditions.

A model transformation engine will then translate this internal representation into eCherry-compatible Modelica simulation models by mapping DSL constructs to corresponding eCherry components. Finally, a Modelica code generator will assemble the necessary simulation structures and configure their parameters according to the DSL specification.

The prototype will likely be implemented using Python for DSL parsing and transformation logic. Generated Modelica models will target the eCherry library and will be executable in standard Modelica simulation environments. Development will proceed in stages beginning with the definition of the

metamodel and DSL syntax, followed by implementation of the parser, transformation engine, and code generator.

Expected Results and Potential Challenges

Expected Results

The project is expected to produce:

- **A prototype DSL-based modeling tool**
  - allows users to define electrochemical reactors using a higher-level modeling language
  - generates simulation-ready models automatically
- **Automatic generation of eCherry simulation models**
  - DSL descriptions transformed into **Modelica code compatible with eCherry**
- **Demonstration of a full model-driven workflow**
  - DSL model
  - metamodel-based representation
  - model transformation
  - generated executable artifact
- **Example reactor models**
  - small laboratory electrolyzer
  - multi-cell reactor configuration
  - other electrochemical reactor examples
- **Comparison of model correctness**
  - generated models will be compared with manually constructed eCherry models
  - simulation outputs should match or closely approximate the results produced by manually written models

- **Reduction in modeling effort**
  - DSL models expected to require fewer lines of specification
  - less manual configuration of simulation components
- **Reduction in modeling time**
  - defining reactor systems using the DSL should take significantly less time than manually assembling Modelica components
- **Improved exploration of design variations**
  - parameters such as electrode area or operating temperature can be easily modified and regenerated

Potential Challenges

Several challenges may arise during development:

- **Designing an effective metamodel**
  - must capture important reactor concepts
  - must remain simple enough for DSL modeling
- **Mapping DSL constructs to eCherry components**
  - transformation rules must correctly generate Modelica structures
- **Ensuring valid Modelica code generation**
  - generated models must be syntactically correct and compatible with eCherry
- **Balancing expressiveness and simplicity**
  - DSL should be powerful enough to represent reactor configurations
  - but simple enough for domain experts to use easily

Example Text Excerpt (AI):

The project is expected to demonstrate that high-level electrochemical reactor models can be automatically translated into executable eCherry simulation models using a model-driven approach. The prototype system will allow users to describe reactor configurations using a domain-specific language and automatically

generate Modelica simulation models compatible with the eCherry framework. Several example reactor configurations, such as a laboratory-scale electrolyzer and a multi-cell reactor system, will be used to evaluate the modeling workflow. Generated models will be compared with manually constructed eCherry models to verify that simulation behavior remains consistent. Additionally, the effort required to define reactor models will be compared by measuring differences in modeling time and specification complexity between manual Modelica modeling and DSL-based modeling.

Potential References:

1. Pyschik, J. F., Ebeling, K. M., Prokopou, G. I., Rix, M. J., Seidenberg, J. R., Kleiner, M. B., ... & Bongartz, D. (2025). eCherry: A Modelica Library for Modular Dynamic Modelling of Electrochemical Reactors. *Electrochemical Science Advances*, *5*(5), e202400030.
    - Directly describes the **simulation framework your project builds on**
    - Provides context for **manual Modelica modeling complexity**
2. Schmidt, D. C. (2006). Model-driven engineering. *Computer-IEEE Computer Society-*, *39*(2), 25.
    - Defines **core MDE concepts**
    - Frequently cited in MDE papers
    - Use to support claims about abstraction, automated artifact generation, and model transofmations
3. Selic, B. (2003). The pragmatics of model-driven development. *IEEE software*, *20*(5), 19-25.
    - Discusses **benefits of raising abstraction levels**
    - Connects models to **automated code generation**
4. Fowler, M., & Languages, D. S. (2010). The Addison-Wesley Signature Series.
    - Classic DSL reference
    - Supports your **DSL design motivation**

5. Fritzson, P. (2015). *Principles of object-oriented modeling and simulation with Modelica 3.3: a cyber-physical approach*. John Wiley & Sons.
    - Authoritative reference for **Modelica modeling**
    - Explains **component-based simulation models**
6. Kelly, S., & Tolvanen, J. P. (2008). Domain-specific modeling: enabling full code generation. John Wiley & Sons.
    - explains **DSL + automatic code generation**
    - directly matches your **DSL → simulation model** pipeline
7. Stahl, T., Völter, M., & Czarnecki, K. (2006). *Model-driven software development: technology, engineering, management*. John Wiley & Sons, Inc..
    - explains **metamodels, models, and transformations**
    - reinforces your modeling pipeline